

Universidade Federal de Minas Gerais
Departamento de Ciência da Computação
Curso de Graduação em Ciência da Computação

Trabalho Prático 2 - Redes de Computadores
Patrulhamento Antidesmatamento

Lucas Affonso Pires
Matrícula: 2023028420

Belo Horizonte, Novembro de 2025

1 Introdução

Este relatório descreve a implementação de um sistema distribuído para o monitoramento e combate ao desmatamento na Amazônia Legal, desenvolvido como parte da disciplina de Redes de Computadores.

O sistema simula a comunicação entre sensores ambientais (Cliente) e uma central de controle (Servidor) utilizando a API de Sockets UDP e a biblioteca de threads POSIX (pthreads) em linguagem C. O objetivo principal é detectar focos de desmatamento em 45 cidades e despachar, de forma automatizada, equipes de drones localizadas nas capitais para averiguação.

2 Arquitetura do Sistema e Protocolo

A arquitetura segue o modelo Cliente-Servidor sobre o protocolo de transporte UDP (User Datagram Protocol). Embora o UDP seja um protocolo não confiável e sem conexão, foi implementada uma camada de aplicação simples para garantir o reconhecimento de mensagens críticas através de ACKs (Acknowledgements).

2.1 Estrutura das Mensagens

Para permitir a troca de diferentes tipos de informações em um único socket, definiu-se um protocolo binário com cabeçalho fixo e payload variável. Para garantir o alinhamento de memória entre diferentes arquiteturas, utilizou-se o atributo `__attribute__((packed))` nas structs.

Os tipos de mensagens definidos no arquivo `common.h` são:

- **MSG_TELEMETRIA (1):** Cliente envia status das 45 cidades.
- **MSG_ACK (2):** Confirmação de recebimento (servidor ou cliente).
- **MSG_EQUIPE_DRONE (3):** Ordem de despacho de drone (Servidor → Cliente).
- **MSG_CONCLUSAO (4):** Notificação de missão finalizada (Cliente → Servidor).

3 Implementação do Servidor

O servidor é responsável por manter o estado global do sistema, incluindo o grafo das cidades, a disponibilidade das equipes de drones e o cálculo de rotas.

3.1 Algoritmo de Roteamento (Dijkstra)

Quando um alerta de desmatamento é recebido, o servidor executa o algoritmo de Dijkstra para encontrar a equipe de drones disponível mais próxima.

A lógica de seleção segue os seguintes critérios:

1. O algoritmo calcula a distância da cidade em alerta para **todos** os outros nós do grafo.
2. O servidor itera sobre a lista de cidades, filtrando apenas aquelas que são **Capitais** (`tipo == 1`).
3. Verifica-se se a equipe da capital está livre (`equipe_ocupada[id] == 0`).
4. Seleciona-se a capital com a menor distância calculada.

3.2 Gerenciamento de Estado

O servidor mantém um vetor `equipe_ocupada` para evitar que uma mesma equipe seja despachada para múltiplos alertas simultaneamente. A equipe só é liberada mediante o recebimento da mensagem `MSG_CONCLUSAO`.

4 Implementação do Cliente

O cliente simula o ambiente distribuído dos sensores e a atuação física dos drones. Para garantir a concorrência e a responsividade, o cliente foi implementado utilizando 4 threads distintas.

4.1 Divisão de Responsabilidades (Threads)

- **Thread de Monitoramento:** Simula a leitura dos sensores. A cada iteração, existe uma chance de 3% de uma cidade entrar em estado de alerta. Utiliza um `mutex` para proteger a escrita no vetor de estados compartilhado.
- **Thread de Telemetria:** A cada 30 segundos, coleta o estado atual de todas as cidades (protegido por `mutex`), empacota os dados e envia ao servidor via UDP.
- **Thread de Recepção:** Escuta a porta UDP continuamente. Ao receber uma ordem de drone (`MSG_EQUIPE_DRONE`), ela envia um ACK imediato e insere a ordem em uma fila de missões, utilizando uma variável de condição (`pthread_cond_signal`) para notificar a thread de simulação.
- **Thread de Drone (Simulação):** Implementa o padrão *Producer-Consumer*. Ela dorme aguardando um sinal da thread de recepção. Ao acordar, consome a missão da fila, simula o tempo de deslocamento (sleep aleatório de 1 a 30s) e envia a mensagem de conclusão ao servidor.

5 Conclusão

O trabalho permitiu a aplicação prática de conceitos fundamentais de redes e sistemas operacionais. A utilização de UDP exigiu o tratamento manual de confirmações (ACKs) na camada de aplicação. O uso de threads no cliente demonstrou a importância de mecanismos de sincronização (`mutex` e variáveis de condição) para evitar condições de corrida e garantir a integridade dos dados compartilhados entre as rotinas de monitoramento e comunicação. O algoritmo de Dijkstra mostrou-se eficaz para a tomada de decisão automatizada no despacho de recursos.